# LELEC2870

## Machine learning Project

*Prof. M. Verleysen, Prof. J. Lee*

**Academic Year : 2025 - 2026**


Louis Baltus *DATS2M*
Nathan Devaux *DATS2M*

# Introduction

The project aims to model, predict and understand the risk of heart failure within ten years among the Smurf population. To achieve these goals, we divided the work into three different parts: linear models, nonlinear models and integrating images into a model. In each of these sections, we will develop how we preprocessed the data, the type of model we chose, why and how we built them. In addition, we will discuss their individual performances and limitations. Finally, we will try to extract some rules from our prediction tools that could help us understand which factors (variables) increase the risk.

# PART 1 : Linear Models

The first part of our project focuses on how we fitted the best linear model to the medical dataset. Our target variable is the risk of developing heart failure within the next ten years. Our objective is to fit a linear model capable of predicting this risk.

## Preprocessing

Before implementing our models, we had to preprocess the dataset to get the best of it. The first step was to remove the last irrelevant column of our dataset, as it was the reference to the corresponded image. After checking for missing or duplicate values due to miss computation, we looked at our different variable distributions, in order to better understand the data. We saw that the different risk where well distributed, we didn't have only smurfs with a high risk or only smurfs with a small risk. We also identified three variables that might impact the risk (cholesterol, blood pressure and weight). For the other variables, it was not easy to distinguish differences between high risk and low risk smurfs. As we were pursuing our exploration of the data we noticed that the blood pressure variable was a bit special. In fact, we have categorized the risk into five groups and saw that the kurtosis of these five groups were far from zero for the blood pressure variable. We know that the log transformation can sometimes be good for smoothing a variable and making it more symmetric, it could also reduce the impact of outliers. We didn't wanted to impose the log-transformation in the model as we don't have problem of time with the optimization of the linear model, we put it in the cross-validation so that the model uses the variable blood pressure or log(blood pressure) variable.

At that time, we had a clear understanding of how the risk was distributed in the dataset, we could start our first transformations. For example, we transformed our categorical variables into dummy variables to make them compatible with our package. At first, we applied the OneHotEncoder() function, but we saw that changing the variable: *sarsaparilla, smurfberry liquor and smurfin donuts* into ordinal values rather than categorical, it improved the RMSE of the model. This made sense, as these variables represent different levels of consumption. We also chose to drop the first column of the OneHotEncoder() function to avoid multicollinearity issues in our model.

Finally, we used a pipeline to avoid data leakage and get some reusability to easily compare the performance of our four linear model with the same transformation. We also tried different standardization method ("z-score, "robust", "min-max" or none). We know that testing four different technics won't significantly change the RMSE of the test set, but as the model are running pretty fast at the moment we have the 'luxury' to include these

kinds of choice in our cross-validation. The last transformation we added in our model is the PolynomialFeatures() function. Here we also made a choice if the model wanted the pure polynomial degree term.

### Model selection

For this section, we chose to explore four models : *a classical linear regression, a ridge regression, a lasso regression and an elastic-net regression.* All of these models has its own particularities, so we had to adapt their hyperparameters individually using Optuna(extension of Cross-Validation using Bayesian statistics). All the model except the linear regression have a penalty term, this is why we have not yet implemented any feature selection as the filters or Wrappers.

Now, that we have preprocessed our data, selected our linear models, and iteratively refining preprocessing and optimization steps with Optuna, we were ready to select our final model. After numerous iterations, the model that best minimizing the RMSE was the Lasso regression with a RMSE of 0.0396 on the training set and 0.04106 on the test set. Finally, as Lasso regression is a linear model, it has a coefficient for every feature. However, after using a second-degree polynomial transformation (PolynomialFeatures), the interpretation of the coefficients became hard individually. But we attempted to interpret the model in PART 4.

## PART 2: Nonlinear models

Then we had to do the same exercise with nonlinear models. Following the same structure as before, we will first discuss how we preprocessed the data for these particular techniques and then describe the models themselves and their performances. An important new part of the design was also choosing the hyperparameters. Indeed, the nonlinear models came with many parameters that we had to tune for each kind of algorithm. We therefore gave each model a domain for every hyperparameter that the optimizer could explore in the hope that it would converge to the optimal set of hyperparameters giving us the best possible performances. As in Part 1, we couldn't test multiple types of standardization for the predictor variables and the risk, as we wanted our optimization to focus primarily on the hyperparameters.

### Preprocessing

Before diving into our model design, it is important to highlight that we had to preprocess the data differently than in the linear part of this project. Even more, we divided our nonlinear models in two categories as they have different needs regarding the data.

The first category regroups the *tree models* : a **decision tree**, a **random forest** and a **XGBoost**. We didn't need to standardize our variables for these models because they all work with splits leaving them insensitive to this transformation. We also didn't need to give them polynomial features as they already dealt with it by themselves. However, we gave our optimizer the opportunity to decide wether the variable *bloodpressure* needed to be log-transformed, as in the linear model. However, for tree models it has not the same benefit as for the linear model. In fact, the purpose of this transformation in the tree model was to reduce the effect of the outliers as the tree models are working with splits.

In addition, we set the number of folds to ten for the models that ran quickly, and to five for those that were slower. We know that increasing the number of folds generally gives more stable performance estimate, which is why we kept a large k for the fast-running models. As in the linear models, tree models don't need feature selection technique that is why we didn't put these kind of methods yet.

The second category is compose of models that required the standardization of the training set variables and the target variable. These are the **support vector regression**, the **multilayer perceptron** and the **k nearest-neighbors**. In addition, unlike the previous cases, where variable selection was automatically dealt by our algorithms, here we had to decide which variables to give to our models. To do so, we implemented three criteria : mutual information, maximum relevance minimum redundancy and forward selection. We could have used other methods, as seen during the classes like the backward method (similar to forward), but we had to make a choice for computational reason and found that those three were more than satisfying and relevant.
In addition, we applied the same treatment to the blood pressure variable (log-transformation). As with the tree models, the transformation could reduce the impact of outliers.

## Tree models

Before diving into our *tree models*, it is important to understand why they belong to the same family and how we expect them to behave. The simplest model is by far the decision tree. It divides the features space sequentially by creating rules like "if feature greater than". It makes it capable of handling non linear relations and really interpretable but it also makes it unstable and likely to overfit if we don't give it the proper hyperparameters.

The random forest tries to correct all these defaults by training numerous independent trees that are then aggregated. It has generally better performances and stability but there's a tradeoff in interpretability.

Then comes the XGBoost which is often the best of the family. His philosophy is different from the previous one. Instead of building independent trees, here we build them sequentially so the next one correct the errors of the prior tree. Combined with regulation processes (L1/L2), this model can apprehend complex patterns but has a lot of hyperparameters to tune.

Thus, we expect the random forest and XGBoost to perform better than the simple tree models, even if in certain cases a tree can be better than a random forest. Finally, we suspect that the XGboost algorithm will be the best one based on what we have just explained.

### Decision tree

In addition to the parameters the we discussed in the preprocessing section, we asked Optuna to focus on four hyperparameters for our decision tree: the minimal sample size on a leaf, the maximum depth of the tree, the minimal sample size to create a split and the maximum number of features considered on each leaf.

By using, Optuna we have the chance to see the importance of each hyperparameter in our models. The importance is computed and based on the functional ANOVA framework, which gives us how the RMSE varies when the model changes one or a combination of

hyperparameter. In this case, Optuna found that the scaling the target y was important, max depth and max feature were the three most important hyperparameter. The Decision tree got an RMSE of 0.04891 on the training set and 0.05445 on the test set. It seems that the Decision tree model has problem to generalize as it lost a lot of RMSE between the training and the test set.

### Random forest

For this model, we choose to optimize 5 hyperparameters : the number of three and the same 4 parameters that we had in the decision tree (the random forest being special combination of decision trees). After 500 iterations via Optuna, the best RMSE score after cross validation (nfolds =10) was 0.0436 and 0.0454 on the test set. For the most important parameters, it appeared that it was the max depth by far, followed by the minimal sample size on each leaf and then by the maximum number of features considered but they were really less contributing. That's all normal as we know that random forest are more robust and less sensitive to the hyperparameters.

### XGBoost

XGBoost had a lot of hyperparameters. Here we considered the number of boosting rounds, the learning rate, the maximum depth, minimum child weight, gamma, the subsample and colsample by tree, lambda and alpha regulation and the tree method.

Once again we chose 10 as number of folds for our cross-validation. Here, it seemed like that gamma was the only really important parameter or at least that it was affecting the performance in another order of magnitude compared to the other ones. The model obtained a RMSE of 0.04025 on the training set and a RMSE of 0.04082 on the test set. The XGBoost generalized well the problem as it's not loosing a lot of performance between the training and in the test set.

It's important to note that, for the moment Optuna has chosen to make a log transformation on all linear model and all tree model, it was an important option to add !

## Support vector regression

With this new algorithm, we leave the tree models class for a model based on the distance: support vector regression (SVR). To be precise, we have implemented two methods, both in a kernel variation: a normal SVR and an extension called NuSVR. The first method is a classical SVR algorithm with a Gaussian kernel. The hyperparameters were the variable selection method, the C (parameter of regulation), the gamma (for kernel width) and the $\epsilon$ for the width of the insensitivity tube. We chose though to fix the polynomial to 2 in an attempt to stay sober in our computation requirement. For this model again, we could afford 10 folds of cross validation so we applied it. Finally, after optimization we observed that our best model was a full one (with all the features) that applied the log-transformation too. Gamma was the most important hyperparameter followed by $\epsilon$.
To go further and as we had some good result with the classical SVR, we tried a variation of the SVR algorithm called the "NuSVR". The difference came from the fact that instead of having to parameters for C and $\epsilon$, this method merged them into a unique new $\nu$ or "nu". The rest of the tuning points being the same and as this declination had a greater RMSE, we won't extend more here on this one.

### MultiLayer Perceptron

For the MLP or multilayer Perceptron, we quickly saw that we had to make strategic choices regarding the domain of each hyperparameter and the number of hyperparameters to tune. Indeed, we had really long computation time for deceiving performances. That's why for this particular model we decreased the number of folds to 5. We also reduced the number of possible architectures, to focus on two : the one-hidden-layer or "shallow network" or the two-hidden-layers or "double-couche". We also gave it the choice of activation function (relu or tanh), $\alpha$ (shrinking factor), the type of learning rate (constant or adaptive) and the initial learning rate. In addition, the MLP doesn't need the polydegree of two so we make it equals to one.
In the end, we obtained a model with 2 hidden layers with respectively 252 and 247 neurons, a relu activation, an $\alpha$ of 0.0032 and a constant learning rate with a 0.0031 value. In addition, the best results were obtained with a log-normalized blood pressure. And for the first time, it was this parameter that had the biggest impact on the model performance, followed by the choice of activation function and the type learning rate (constant here).

### K Nearest Neighbors

For the last nonlinear model, we chose to implement a K nearest neighbors algorithm which is a totally different kind of model. Indeed, this one looks at the k nearest observations from the given point and tries to estimates the value of the target variable based on theses points. Of course, it also came with its own hyperparameters. The most obvious one is k, the number of observations that we look at for the estimation. But we also chose to optimize whether or not we gave particular weights to our neighbors based on a distance criterion and the criterion itself. In addition we also asked the optimizer to choose between different algorithms of neighbor research (auto, kd-tree, ball-tree, brute) and the leaf size associated to some of these algorithms (kd-tree, ball-tree). The model were really hard to optimize, as we tried a lot of different hyperparameters but the model was not performing as it obtains a RMSE of 0.526 on the training set and 0.05431 on the test set.

## PART 3 : Image integration

For this last section about model implementation, we had to push our algorithms deeper one last time. This time the improvement didn't came from a new technique or at least not directly. Indeed, thanks to an additional source of information, the 48×48-pixel heart scans captured by a MRI-machine, we could create a new numerical variables to enhance our model performances.

### CNN Preprocessing

But the creation of this new vector didn't come from nowhere. In fact we had to create a CNN or convolutional neural network. Our CNN was composed of a double convolution then max-polling layers (4 in total), followed by two fully connected layers. However this CNN also had to be optimize like any other model of this work. In order to get the best one, we chose to focus on the number of features on the first fully connected layer, the number of epochs, the batch size or number images we give to our model during an epoch (a size of 8 was chosen by the optimizer) and finally the learning rate. At the end of the

process, we extract 32 new features. It was a possibility to change the number of features that we extract from the CNN, as we did, but the model were losing a bit of performance that is why we now stick to 32.

### Model selection

Since we had our new variable, it was time to see if giving it to some of our models would have an impact on their RMSE scores. We picked the XGBoost as it was the most performing model from the two previous part.By adding the new features in the dataset it made our XGBoost more performing as it obtains a RMSE of 0.0307 on the training and 0.0357 on the test set.

## PART 4 : Interpretation

It's not possible to interpret all the model in this works, that is why we interpreted our best linear model and our best non linear model. The Lasso regression and the XGboost in the multi modal problem and the classical XGboost were our three best models. In Figure 1 and Figure 2 the goal was to look at the residuals for each continuous and categorical variables in the lasso regression. We also performed the same plots for the XGboost in the multi modal situation. In these visualization it's not possible to clearly see some pattern where our models are making some errors. That is why we created Figure 5 and Figure 6 where we crossed the most relevant variables. In these visualizations, we didn't show the error, but what makes someone at risk or not. The darker the zone is, the less risks it means. In our case, the more you go to the right corners to more risk you get. It means that having an extreme value for the potassium variable implies a large risk, and almost the same conclusion for the hemoglobin variable. By crossing this result with the table 4, we notice that the smurf with low risk are close to the mean for the potassium variable, and the smurf at risk are far from the mean.

This easy comparison between table 3 and 4 is really interesting and we clearly see how the two models adapts to the variable. For exemple, the people have more weight in the top 5% risky individuals in the XGboost model. This means that the model put more weight on this variable than the Lasso Model. Same thing for the variable hemoglobin. Another interesting difference is that the age of low risk people in Lasso model has a mean of 90 where it has a mean of more than 100 in XGboost, meaning that for XGboost being older is more determinant in the risk of having heart problem than in the Lasso Model. These interpretations should be treated cautiously. Of course the XGboost model has more information with the help of the image, but it seemed relevant to notice it.

## Conclusion

Finally, our results showed a clear progression. When simple linear models gave us solid bases, more robust methods quickly took the lead. Moreover, XGBoost won the conquest and its performances improved even further as we gave it the features built with the CNN. In the end, the multimodal XGBoost model got the best results. It highlights that combining medical data with imaging really were relevant when predicting long-term heart-failure risk in Smurfs.

# Annexe

Table 1: Performances of the models

| **Model** | **Train** | **Test** | **Image?** |
|---|---|---|---|
| LinearRegression | 0.04004 | 0.04115 | No |
| LassoRegression | 0.03953 | **0.04106** | No |
| RidgeRegression | 0.03984 | 0.04115 | No |
| ElasticNetRegression | 0.03954 | 0.04108 | No |
| DecisionTree | 0.04891 | 0.05445 | No |
| RandomForest | 0.04355 | 0.04553 | No |
| XGBoost | 0.04026 | **0.04083** | No |
| SVR | 0.0392 | 0.04114 | No |
| MLP | 0.0434 | 0.04493 | No |
| KNN | 0.0526 | 0.05431 | No |
| NuSVR | 0.04101 | 0.04269 | No |
| XGBoost | 0.0307 | 0.0357 | Yes |

Table 2: Top 12 coefficient in absolute value (Lasso Regression)

| **Feature** | **Coefficient** |
|---:|---|
| potassium$^2$ | 0.356131 |
| hemoglobin$^2$ | 0.265432 |
| age | 0.201924 |
| height weight | -0.195108 |
| weight | 0.187187 |
| blood pressure$^2$ | 0.172278 |
| hemoglobin | -0.171084 |
| blood pressure | 0.146739 |
| height | -0.142833 |
| cholesterol | 0.123378 |
| weight$^2$ | 0.103890 |
| potassium | 0.098681 |

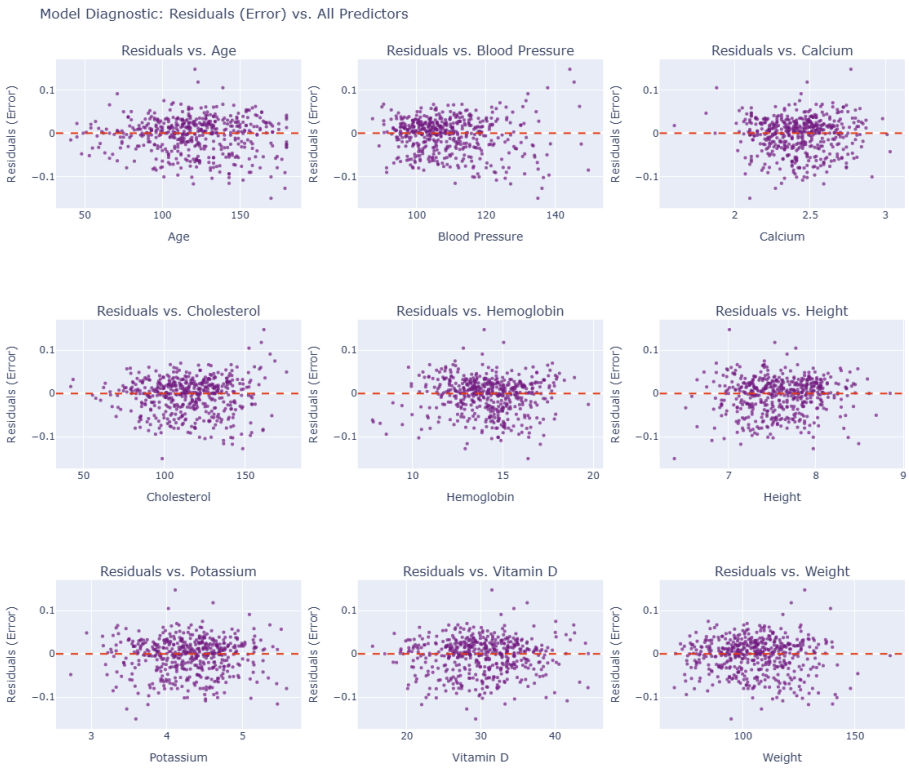


Figure 1: Error caracterisation Lasso Regression

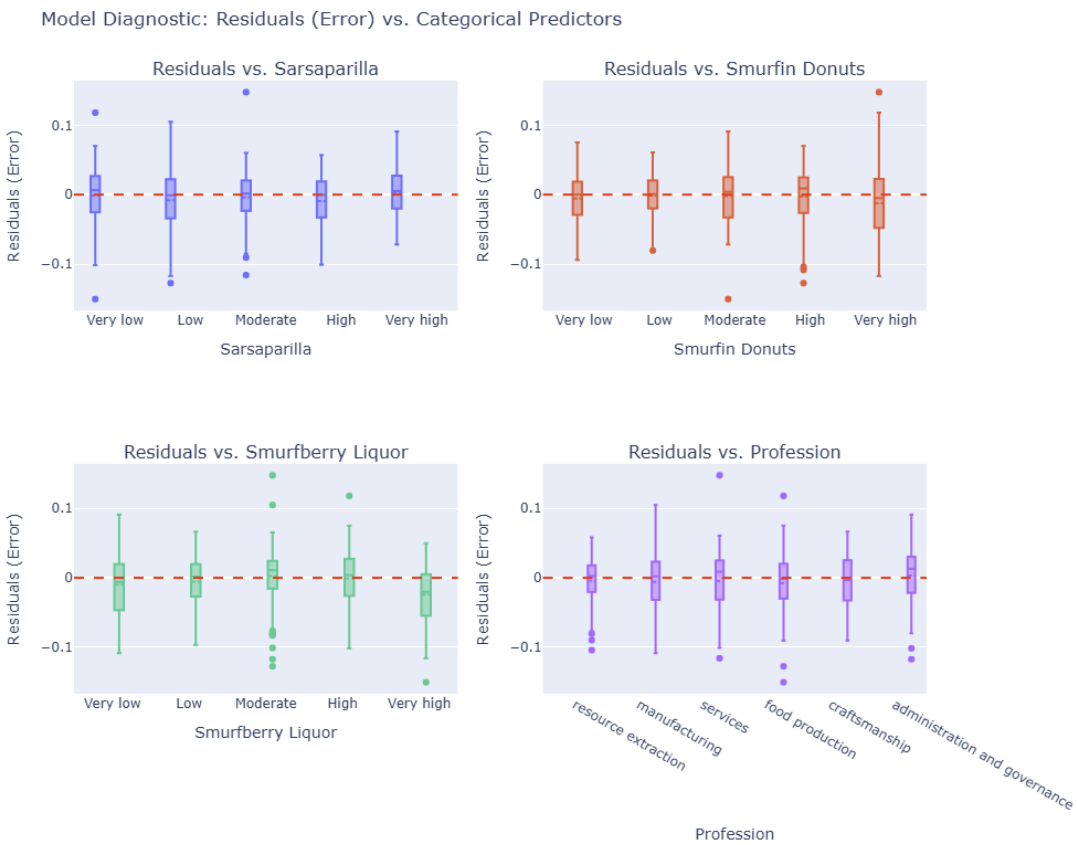


Figure 2: Error caracterisation Lasso Regression

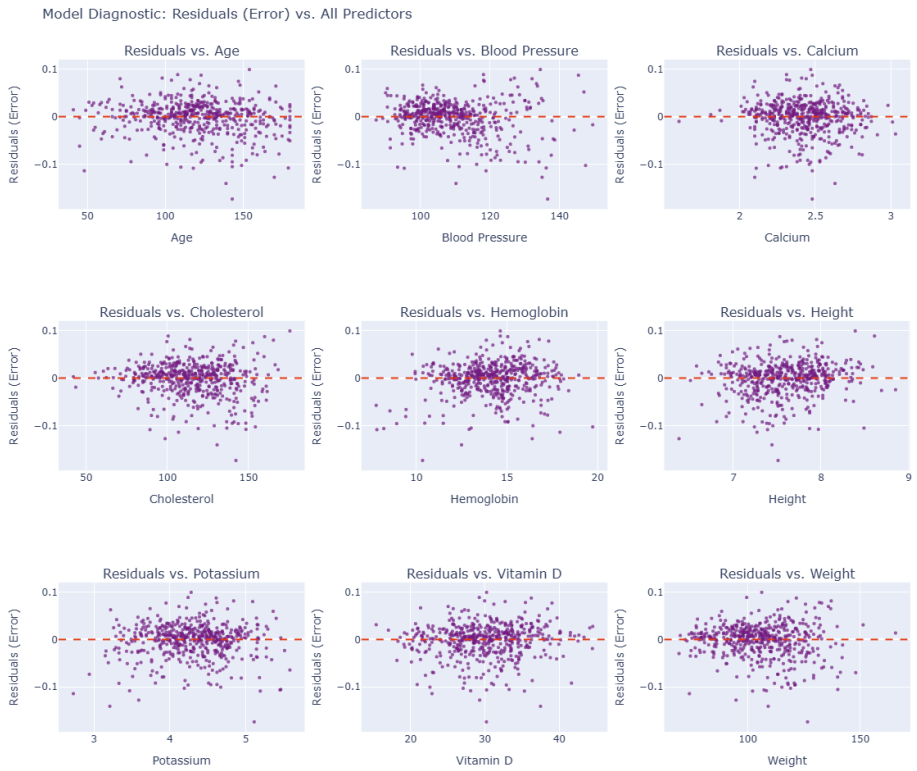


Figure 3: Error caracterisation XGBoost

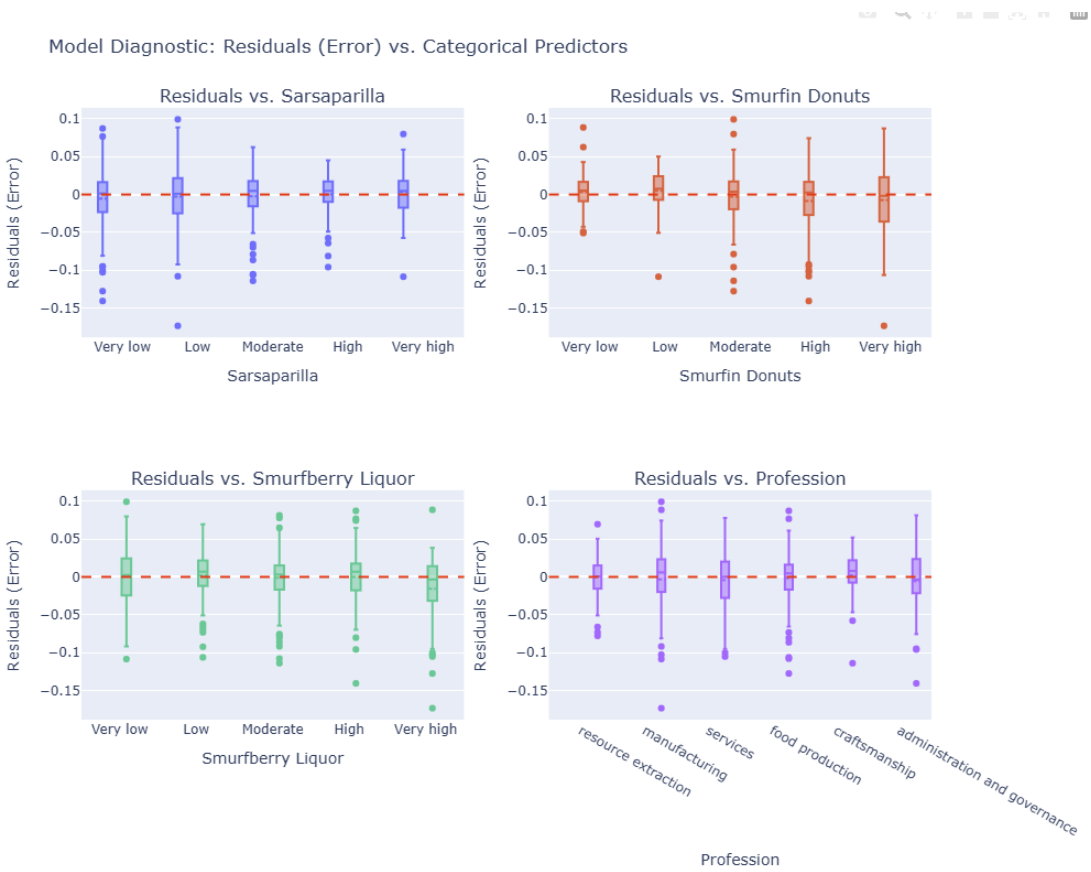


Figure 4: Eroor caracterisation XGBoost

Table 3: Characteristics of Smurf in different groups in the Lasso Model

| **Characteristics** | **Mean Top 5% at risk** | **Mean Bottom 5% at low risk** | **Mean test_set** |
|---|---|---|---|
| age | 129.48 | 90.00 | 120.16 |
| blood pressure | 131.17 | 103.39 | 109.04 |
| calcium | 2.44 | 2.43 | 2.43 |
| cholesterol | 145.81 | 103.78 | 114.91 |
| hemoglobin | 12.87 | 14.37 | 14.16 |
| height | 7.33 | 7.72 | 7.59 |
| potassium | 4.29 | 4.31 | 4.30 |
| vitamin D | 30.51 | 27.61 | 30.02 |
| weight | 118.89 | 96.24 | 105.51 |

Table 4: Characteristics of Smurf in different groups in the XGBoost multi modal

| **Characteristics** | **Mean Top 5% at risk** | **Mean Bottom 5% at low risk** | **Mean test_set** |
|---|---|---|---|
| age | 129.96 | 101.44 | 120.16 |
| blood pressure | 132.31 | 100.49 | 109.03 |
| calcium | 2.40 | 2.44 | 2.42 |
| cholesterol | 144.59 | 100.55 | 114.90 |
| hemoglobin | 14.18 | 14.53 | 14.16 |
| height | 7.41 | 7.65 | 7.59 |
| potassium | 4.19 | 4.34 | 4.30 |
| vitamin D | 30.05 | 28.62 | 30.02 |
| weight | 123.91 | 95.33 | 105.50 |

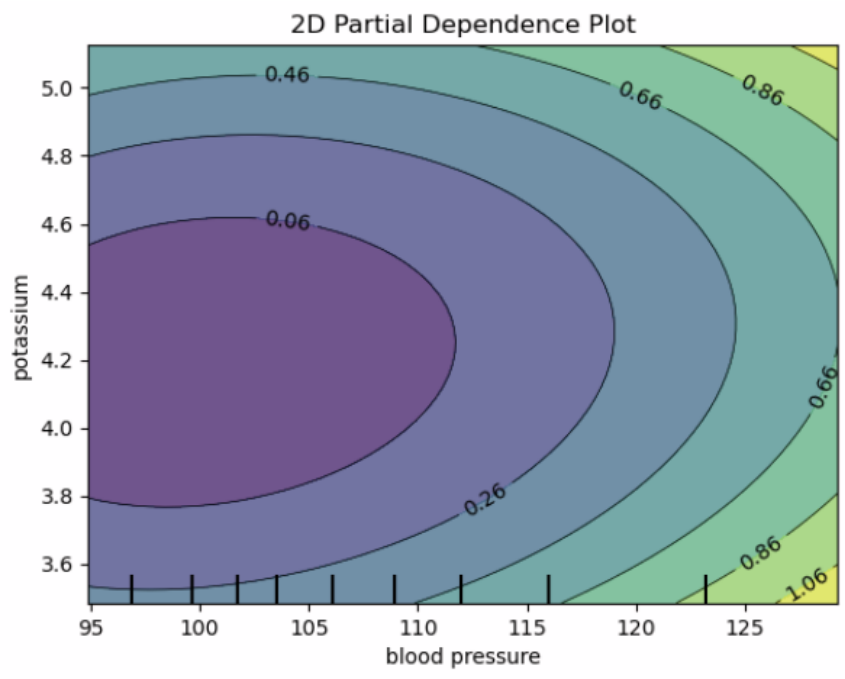


Figure 5: Partial plot Lasso

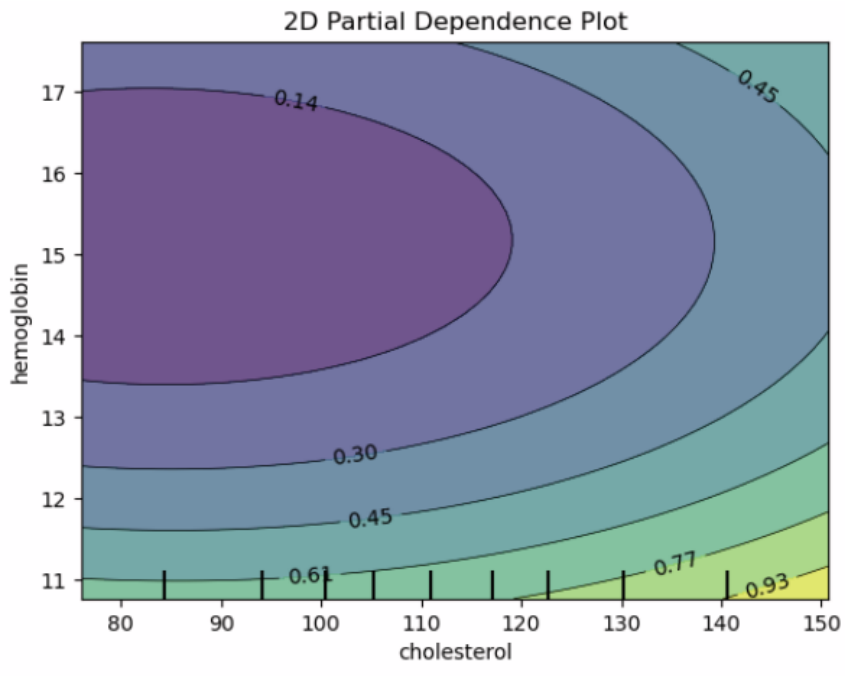


Figure 6: Partial plot Lasso